# MONGODB HANDBOOK

## Page 1: MongoDB Fundamentals

### ☆ 1. What is MongoDB?

- MongoDB is a NoSQL, open-source document database.
- It stores data in flexible, JSON-like documents with dynamic schema.
- Designed for high performance, scalability and easy development.

### ☆2. SQL vs NoSQL

- SQL databases have fixed schema (tables, rows).
- NoSQL databases have dynamic schema.
- MongoDB is schema-less, so fields can vary from document to document.

### ☆3. Why MongoDB?

- Schema-less (flexible structure)
- Easy to scale (horizontal scaling)
- High performance
- Supports complex queries
- Rich query language
- Perfect for real-time applications
- Open source and widely used.

### 4. MongoDB Data Model (Hierarchy)

Database → A container for collections.

↓

Collection → A group of MongoDB documents.

↓

Document → A set of key-value pairs (BSON format).

↓

```
{
  _id : ObjectId("64f2a1b3e7"),
  name : "Aman",
  age : 25,
  email : "aman@gmail.com",
  skills : ["Java", "Selenium", "SQL"],
  address : {
      city : "Gurugram",
      state : "Haryana",
  }
}
```

→ Example of a MongoDB Document

MongoDB stores data in BSON (Binary JSON) format which is more efficient than JSON.

### ☆5. BSON

- BSON stands for Binary JSON.
- It is the internal data format used by MongoDB.
- Supports more data types than JSON.
- More efficient for storage and transmission.

### ☆6. Key Terms

| | | |
|---|---|---|
| Database | → | Holds collections. |
| Collection | → | Group of documents (similar to table in SQL). |
| Document | → | A record in a collection (similar to row in SQL). |
| Field | → | A key in a document (similar to column in SQL). |
| _id | → | Unique identifier for each document (automatically generated if not provided). |

### ☆7. SQL vs MongoDB (Quick Comparison)

| Feature | SQL (Relational DB) | MongoDB (NoSQL DB) |
|---|---|---|
| Database | Database | Database |
| Table | Table | Collection |
| Row | Row | Document |
| Column | Column | Field |
| Schema | Fixed Schema | Dynamic / Schema-less |
| Data Format | Structured (Tables) | Semi-structured (BSON Documents) |
| Relationship | Joins | Embedded Documents / References |
| Scalability | Vertical Scaling | Horizontal Scaling (Sharding) |

**Pro Tip:** Use MongoDB when your data is changing frequently and you need flexible, scalable and high-performance database.

# MONGODB HANDBOOK

## Page 2 : CRUD Operations

**CRUD Summary**

| | | |
|---|---|---|
| Create | → | insert() |
| Read | → | find() |
| Update | → | update() |
| Delete | → | delete() |

★ CRUD stands for **Create**, **Read**, **Update** and **Delete**.
These are the basic operations we perform on data in MongoDB.

### 1. CREATE – Insert Data

- **insertOne()** – Insert a single document

```
// Insert One Document
db.users.insertOne({
  name : "Aman",
  age  : 25,
  email : "aman@gmail.com",
  skills : ["Java", "Selenium", "SQL"],
  active : true
})
```

- **insertMany()** – Insert multiple documents

```
// Insert Many Documents
db.users.insertMany([
   { name: "Karan", age: 24,
     email : "karan@gmail.com" },
   { name: "Sagar", age: 27,
     email : "sagar@gmail.com" }
])
```

**Sample Collection (users)**

users

```
{
 _id : ObjectId("64f2a1b3e7"),
 name : "Aman",
 age : 25,
 email : "aman@gmail.com",
 skills : ["Java","SQL"],
 active : true
}
{
 _id : ObjectId("64f2a1b3e8"),
 name : "Karan",
 age : 24,
 email : "karan@gmail.com",
 skills : ["Python", "API"],
 active : true
}
...
```

### 2. READ – Retrieve Data

- **find()** – Get all documents

```
// Get all documents
db.users.find()

// Filter with condition
db.users.find({ age: { $gt: 20 } })
```

**Other Helpful Options**

- sort({ age: 1 }) → Ascending
- sort({ age: -1 }) → Descending
- limit(5) → Limit results
- skip(10) → Skip first 10 docs

- **findOne()** – Get first matching document

```
// Get first document
db.users.findOne()
// With condition
db.users.findOne({ name: "Aman" })
// Projection (select fields)
db.users.find({ age: { $gt: 20 } }, {
  name: 1, email: 1, _id: 0
})
```

**Projection:**

1 ⇒ include field
0 ⇒ exclude field
(_id is included by default )

### 3. UPDATE – Modify Data

- **updateOne()** – Update first matching document

```
// Update One
db.users.updateOne(
  { name: "Aman" },                  // filter
  { $set: { age: 26, active: false } }  // update
)
```

- **updateMany()** – Update all matching documents

```
// Update Many
db.users.updateMany(
  { active: true },
  { $set: { active: false } }
)
```

**Update Operators (Top Used)**

| | | | | | |
|---|---|---|---|---|---|
| $set | → Set new value | $inc | → Increment value | $pull | → Remove from array |
| $unset | → Remove a field | $push | → Add to array | $addToSet | → Add unique value to array |

### 4. DELETE – Remove Data

- **deleteOne()** – Delete first matching document

```
// Delete One
db.users.deleteOne({ name: "Aman" })
```

- **deleteMany()** – Delete all matching documents

```
// Delete Many
db.users.deleteMany({ active: false })
```

★ **Important Notes**

- Every document in MongoDB has a unique _id field (ObjectId) automatically.
- Field order does not matter.
- Data types can be different in different documents within the same collection.

**Common Data Types**

String, Int, Double, Boolean, Date, Array, Object, Null, ObjectId, Binary, etc.

# MONGODB HANDBOOK

## Page 3 : Querying & Indexing

### ★ 1. QUERYING BASICS

MongoDB provides rich queries to filter and retrieve data from collections.

| Operation | Description | Example |
|---|---|---|
| find() | Get all documents | db.users.find() |
| find({}) | Filter documents | db.users.find({ age: 25 }) |
| findOne() | Get first matching document | db.users.findOne({ name: "Aman" }) |
| Projection | Select specific fields | db.users.find({}, { name: 1, email: 1, _id: 0 }) |
| sort() | Sort result | db.users.find().sort({ age: -1 }) // Desc |
| limit() | Limit number of results | db.users.find().limit(5) |
| skip() | Skip n documents | db.users.find().skip(10) |
| countDocuments() | Count documents | db.users.countDocuments({ age: { $gt: 20 } }) |

### 2. COMMON QUERY OPERATORS

**Comparison Operators**

- $eq → Equal
- $ne → Not Equal
- $gt → Greater Than
- $gte → Greater Than or Equal
- $lt → Less Than
- $lte → Less Than or Equal
- $in → In array
- $nin → Not in array

**Logical Operators**

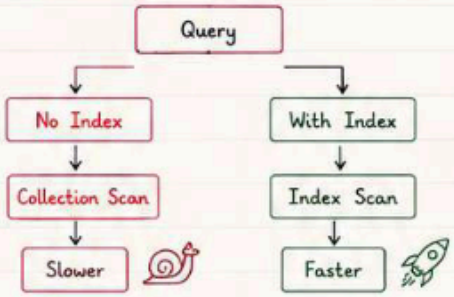
- $and → Logical AND
- $or → Logical OR
- $not → Logical NOT

**Element Operators**

- $exists → Field exists or not
- $type → Match by data type

### 3. INDEXING IN MONGODB

- Indexes improve the speed of read operations.
- Without index, MongoDB has to scan all documents.
- With index, MongoDB can find data quickly.
- Indexes consume extra storage but improve performance.




### 4. TYPES OF INDEXES

| Index Type | Description |
|---|---|
| Single Field Index | Index on one field. |
| Compound Index | Index on multiple fields. |
| Unique Index | Ensures all values are unique. |
| Text Index | For text search. |
| Hashed Index | Hashes the value of a field. |
| TTL Index | Automatically removes documents after a time. |
| Sparse Index | Index only documents that contain the field. |

### 5. CREATING INDEXES (EXAMPLES)

```
// Single Field Index
db.users.createIndex({ age: 1 })      // 1 → Ascending
db.users.createIndex({ age: -1 })     // -1 → Descending

// Compound Index
db.users.createIndex({ age: 1, name: 1 })

// Unique Index
db.users.createIndex({ email: 1 }, { unique: true })

// Text Index
db.users.createIndex({ name: "text", skills: "text" })

// TTL Index  (expire documents after 3600 seconds)
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

### 6. EXPLAIN QUERY

Used to see how MongoDB executes a query.

```
db.users.find({ age: 25 }).explain("executionStats")
```

↓ Shows query plan ↓ Performance stats

**PRO TIP**

- Create index on fields frequently used in queries, sort, filter and join.
- Don't create too many indexes.
- Use explain() to analyze slow queries.
- Regularly review unused indexes.

★ Note: Indexes make reads faster but can slow down writes (insert/update/delete). Choose indexes wisely based on your application.

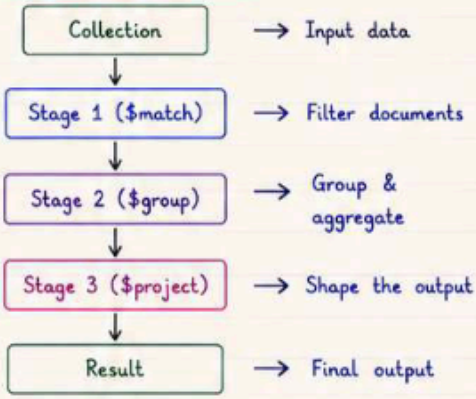
# MONGODB HANDBOOK

## Page 4 : Aggregation Framework

### ★ 1. What is Aggregation?

- Aggregation is a powerful framework in MongoDB used to process data records and return computed results.
- It works on the concept of PIPELINE.
- A pipeline is a sequence of stages , where the output of one stage becomes the input of the next stage.

**Aggregation Pipeline Flow**




### 2. Common Aggregation Stages

| Stage | Description |
|---|---|
| $match | Filters documents (like WHERE in SQL) |
| $group | Groups documents and performs aggregations |
| $project | Reshapes the documents (select fields, rename, calculations) |
| $sort | Sorts the documents |
| $limit | Limits the number of documents |
| $skip | Skips the specified number of documents |
| $lookup | Performs JOIN with another collection |
| $unwind | Deconstructs an array field |
| $count | Counts the number of documents |
| $facet | Allows multiple pipelines in a single query |

### 3. Sample Aggregation Example

```
// Find total number of users per city
db.users.aggregate([
   { $match: { active: true } },
   { $group: { _id: "$city", totalUsers: { $sum: 1 } } },
   { $sort: { totalUsers: -1 } },
   { $project: { _id: 0, city: "$_id", totalUsers: 1 } }
])
```

**Sample Data (users collection)**

```
{
{ "_id": ObjectId("64f2a1b3e7"), "name": "Aman",  "city": "Delhi",     "active": true },
{ "_id": ObjectId("64f2a1b3e7"), "name": "Karan", "city": "Mumbai",    "active": true },
{ "_id": ObjectId("64f2a1b3e9"), "name": "Sagar", "city": "Delhi",     "active": true },
{ "_id": ObjectId("64f2a1b3ea"), "name": "Ravi",  "city": "Bangalore", "active": false },
{ "_id": ObjectId("64f2a1b3eb"), "name": "Neha",  "city": "Mumbai",    "active": true }
}
```

### 4. Stage Details with Examples

**$match (Filter)**

Filters documents that match the given condition.

```
{ $match: { age: { $gte: 18 }} }
```

**$group (Group & Aggregate)**

Groups by a field and performs aggregation.

```
{ $group: {
    _id: "$city",
    totalUsers: { $sum: 1 }
} }
```

**$project (Reshape Output)**

Includes, excludes or computes fields.

```
{ $project: {
    _id: 0,
    city: "$_id",
    totalUsers: 1
} }
```

**$sort (Sort Results)**

Sorts the documents.

```
{ $sort: { totalUsers: -1 } }
```

1 => Ascending ,
-1 => Descending

**$limit (Limit Results)**

Limits the number of documents.

```
{ $limit: 10 }
```

**$skip (Skip Results)**

Skips the specified number of documents.

```
{ $skip: 20 }
```

**$count (Count Results)**

Counts the total documents in the pipeline.

```
{ $count: "total" }
```

### 5. Key Points

- Aggregation is very powerful for reporting and analytics.
- It can replace complex queries and multiple collections processing.
- Used in dashboards, charts, reports, data analysis, etc.
- Always try to start pipeline with $match for better performance.

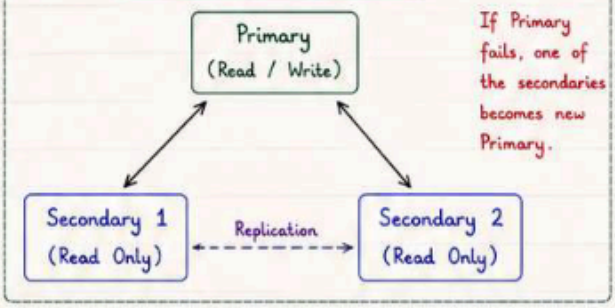
# MONGODB HANDBOOK

## Page 5 : Replication, Sharding & Transactions

### ★ 1. Replication (Replica Set)

- Replication provides high availability and data redundancy.
- Data is copied to multiple nodes (replicas).
- One node is Primary (handles writes).
- Others are Secondaries (replicate data).
- If Primary fails, a Secondary is elected as new Primary (automatic failover).

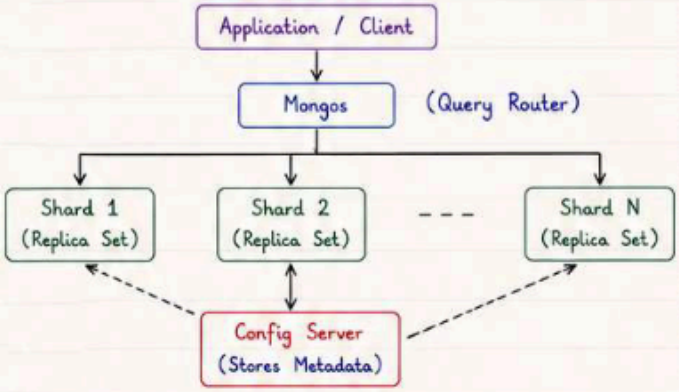
**Replica Set Architecture**




### 2. Sharding (Horizontal Scaling)

- Sharding distributes data across multiple machines.
- Helps in handling large amount of data and high throughput.
- Key Components:
  - Shard : Stores actual data.
  - Config Server : Stores metadata about clusters.
  - Mongos : Acts as query router.

**Sharding Architecture**




### 3. Transactions

- MongoDB supports ACID transactions (since v4.0).
- Transactions allow multiple operations to succeed or fail as a single unit.
- Useful for maintaining data consistency.

**ACID in MongoDB Transactions**

| | |
|---|---|
| A - Atomicity | : All or nothing |
| C - Consistency | : Data remains consistent |
| I - Isolation | : Operations are isolated |
| D - Durability | : Committed data is permanent |

**Example : Transaction in MongoDB (Shell)**

```
const session = db.getMongo().startSession();
session.startTransaction();
try {
    db.accounts.updateOne({ _id: 1 }, { $inc: { balance: -100 } }, { session: session });
    db.accounts.updateOne({ _id: 2 }, { $inc: { balance: 100 } }, { session: session });
  session.commitTransaction();
  print("Transaction Committed Successfully");
} catch (error) {
  session.abortTransaction();
  print("Transaction Aborted: " + error);
} finally {
  session.endSession();
}
```

### 4. Read Concern & Write Concern (Brief)

**Read Concern**

- Defines the level of isolation for read operations.
- Options:
  - local : Default, may read uncommitted data.
  - majority : Reads data that has been acknowledged by majority.
  - snapshot : Provides snapshot isolation.

**Write Concern**

- Defines how many nodes must acknowledge a write.
- Options:
  - w:1 : Acknowledged by primary.
  - w:"majority" : Acknowledged by majority of nodes.
  - w:0 : No acknowledgment (faster but risky).

★ Pro Tip: Use replication for high availability, sharding for horizontal scaling, and transactions for data consistency.

★ Important: Replica Set is required to use Transactions in MongoDB.

# MONGODB HANDBOOK

## Page 6 : Spring Boot Integration with MongoDB

### 1. Add Dependency (Maven)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

### 2. Application Properties

```
spring.data.mongodb.uri=mongodb://localhost:27017
spring.data.mongodb.database=mydb
# mongodb://<username>:<password>@localhost:27017/mydb
# For MongoDB Atlas, use the connection string provided.
```

### 3. Entity Class

```
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
@Document(collection = "users")
public class User {
    @Id
    private String id;
    private String name;
    private int age;
    private String email;

    // Getters and Setters
}
```

- @Document : Marks this class as a MongoDB document and maps it to the "users" collection.
- @Id : Represents the primary key (_id field in MongoDB).

Sample Document in MongoDB

```
{
  "_id" : ObjectId("664f2a1b3e7"),
  "name" : "Aman",
  "age" : 25,
  "email" : "aman@gmail.com"
}
```

### 4. Repository Interface

```
import org.springframework.data.mongodb.repository.MongoRepository;

public interface UserRepository
        extends MongoRepository<User, String> {
    // Custom Queries (if needed)
    List<User> findByName(String name);
}
```

MongoRepository provides:

- save(S entity) – Insert / Update
- findById(ID id) – Find by Id
- findAll() – Get all documents
- deleteById(ID id) – Delete by Id
- count() – Count documents
- existsById(ID id) – Check existence

### 5. Service Layer (Example)

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public User saveUser(User user) {
        return userRepository.save(user);
    }
    public List<User> getAllUsers() {
        return userRepository.findAll();
    }
    public User getUserById(String id) {
        return userRepository.findById(id).orElse(null);
    }
    public void deleteUser(String id) {
        userRepository.deleteById(id);
    }
}
```




### 6. Controller Layer (Example)

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @Autowired
    private UserService userService;
    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.saveUser(user);
    }
    @GetMapping
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }
    @GetMapping("/{id}")
    public User getUserById(@PathVariable String id) {
        return userService.getUserById(id);
    }
    @DeleteMapping("/{id}")
    public void deleteUser(@PathVariable String id) {
        userService.deleteUser(id);
    }
}
```

### 7. Key Annotations

| Annotation | Purpose |
|---|---|
| @Document | Marks the class as a MongoDB document and maps to collection. |
| @Id | Represents the primary key (_id) of the document. |
| @Field | Maps a field to a different name in the document. |
| @Repository | Marks the interface as a repository for data access. |
| @Autowired | Injects the dependency automatically. |

**Tips**

- MongoDB stores data in JSON-like documents (BSON).
- _id is automatically generated if not provided.
- Collections are created automatically when first document is inserted.
- Use indexes for better query performance.